

TIC-ONE Middleware

Complete Technical Analysis & Technology Inventory

Project	tic_one_middleware (v1.0.0+1)
Type	REST backend / middleware – movie ticketing + intercity bus booking + wallet
Language / Runtime	Dart ≥ 3.11 (AOT-compiled native executable)
Framework	Dart Frog (file-system routing)
Database	PostgreSQL (SSL required)
Repository	git, branch main, 24 commits (2026-09-24 → 2026-10-06)
Analysis date	2026-10-07

1. Executive Summary

TIC-ONE Middleware is the server-side API for a consumer ticketing app ("TicOne"). It exposes three product areas over one HTTP/JSON service: **movie booking** (browse, show listings, seat map, seat locking, booking, cancellation), **bus booking** (search, seat map, seat recommendation, holds, promo codes, payment intent, cancellation with tiered refund) and a **wallet** (balance top-up, CinePoints loyalty, Royal Pass premium subscription). A separate **admin API** (/api/admin/*) provides back-office CRUD, dashboard, payments/refunds and audit logs.

The service is a thin, stateless Dart application: each route file delegates to a static service class, which opens a PostgreSQL connection, runs hand-written SQL and returns JSON. There is no ORM, no cache and no message queue.

Key numbers

Metric	Value
Dart source (lib, routes, bin, tool)	≈ 16,800 lines
Service layer (lib/services)	≈ 12,600 lines across 36 files
Route files (routes/)	91 (≈ 20 public groups + 19 admin resources)
Largest services	bus_booking (1,497), bus_service (1,283), wallet (904)
Database tables	14 movie/auth + 15 bus + wallet/admin tables created at runtime or by tools
Runtime dependencies	6 declared (+ transitive crypto)
Automated tests	1 file (CORS/logging middleware), 3+ cases

2. Technology Stack (what is used)

Layer	Technology	Purpose / where
Language	Dart 3.11 (sdk ^3.11.0)	Entire backend, tooling scripts
Web framework	dart_frog ^1.1.0	File-based routing (routes/), middleware, Response.json; generated bin/server.dart under build/
Database	PostgreSQL via postgres ^3.5.16	Connection.open with SslMode.require; named parameters (Sql.named) for all queries
Password hashing	bcrypt ^1.2.0	BCrypt.hashpw / gensalt / checkpw (register, login, reset)
Access tokens	dart_jsonwebtoken ^3.4.1	HS256 JWT, 15 min, claims: sub=userId, email, type=access, issuer
Refresh/reset tokens	crypto (transitive) – SHA-256	32 random bytes (Random.secure, base64url); only the SHA-256 hash is stored
Config	dotenv ^4.2.0	lib/config/env.dart – .env + platform env, fail-fast on missing keys
Formatting	intl ^0.19.0	Date/number formatting
Linting	dart_frog_lint ^0.1.0, analysis_options.yaml	Static analysis
Testing	test ^1.0.0, mocktail ^1.0.0	Unit test of middleware
Packaging	Docker multi-stage	dart:3.11.0 build → dart compile exe → debian:bookworm-slim + ca-certificates, EXPOSE 8080
Logging	Custom AppLogger	Timestamped print() to stdout (info / warning / error)
Payments	Simulated (UPI intent string + gateway key placeholder)	No real gateway SDK integrated
Dev tooling	VS Code launch/settings, tool/*.dart	Migrations, seeding, DB inspection, API smoke tests

3. Architecture

Request flow:

```
Client (mobile / web / admin panel)
  | HTTPS JSON
  v
Dart Frog HTTP server (bin/server.dart, PORT env, default 8080)
  | routes/_middleware.dart -> CORS, OPTIONS 204, request/response timing logs
  v
Route handler (routes/**/*.dart) -> method switch (405 otherwise), param extraction
  v
Service class (lib/services/**) -> validation, auth, business rules, SQL
  v
openDatabaseConnection() -> new PostgreSQL connection per request (closed in finally)
  v
PostgreSQL (SSL)
```

Layering

- **Routes** are intentionally tiny: switch on `HttpMethod` and call `XService.method(context)`.
- **Services** are static classes (no DI, no interfaces). Each handler owns its connection lifecycle and returns a `Response`.
- **Auth utilities** (`AuthUtils`) centralise Bearer parsing, JWT create/verify, token hashing and safe JSON reading.
- **Standalone tools** (`tool/*.dart`) reuse `lib/database.dart` for migrations, seeding and diagnostics.

Directory map

Path	Content
<code>bin/server.dart</code>	Generated Dart Frog entry point (build output copy; routing table)
<code>routes/_middleware.dart</code>	Global middleware: CORS (*), preflight, logging
<code>routes/auth, bookings, buses, bus-bookings, bus-payments, bus-promos, home, movies, promos, royal-pass, shows, theaters, wallet</code>	Public / user endpoints
<code>routes/api/admin/**</code>	Admin endpoints (19 resource groups)
<code>lib/config/env.dart</code>	Typed env access
<code>lib/database.dart</code>	Connection factory
<code>lib/services/{auth,bookings,bus,home,movies,promos,seats,theaters,wallet,admin}</code>	Business logic
<code>db/</code>	SQL schemas (02 movie, 04 bus), seed data, full dumps (30-09, 01-10, 03-10, 04-10), theaters.csv
<code>tool/</code>	19 dev scripts (migrate, seed, inspect, API tests)
<code>test/</code>	Middleware unit tests
<code>API_DOCUMENTATION.md</code>	1,090-line frontend integration guide

4. API Surface

4.1 Public / user endpoints

Area	Endpoints	Notes
Auth	POST /auth/register, /login, /logout, /refresh-token, /forgot-password; GET /auth/profile	Login accepts email or phone; profile includes Royal Pass membership
Home	GET /home	Banners, trailers, movies feed
Movies	GET /movies, /movies/<id>, /movies/<id>/shows; GET /theaters	City / search filtering (slug route removed in latest commit)
Seats	GET /shows/<id>/seats; POST /shows/<id>/lock-seats	8-minute lock, returns lockToken
Movie bookings	POST /bookings/create; GET /bookings/my-tickets, /upcoming, /past, /cancelled, /<code>; POST /bookings/cancel, /<code>/cancel	Booking code TIC-YYYY-NNNNNN; QR payload
Promos	GET /promos; POST /promos/validate	Movie promo codes
Bus search	GET /buses, /buses/search, /buses/cities, /buses/routes, /buses/<id>, /seats, /boarding-points, /dropping-points	busId or tripId accepted
Bus seats	POST /buses/<id>/hold-seats, /bus-bookings/hold-seats; GET POST /buses/recommend-seats	10-minute hold; heuristic seat recommender
Bus booking	POST/GET /bus-bookings; GET /bus-bookings/<id>; POST /<id>/cancel; GET /<id>/refund	Lookup by id, booking code or PNR
Bus payments	POST /bus-payments/create, /verify; GET /bus-payments/<id>	Payment intent lifecycle
Bus promos	GET /bus-promos; POST /bus-promos/validate	Percentage / flat, min amount, max cap
Wallet	GET /wallet; POST /wallet/topup, /pay, /convert-points; GET POST /royal-pass, /royal-pass/purchase, /wallet/royal-pass	Balance, CinePoints, subscription

4.2 Admin endpoints (/api/admin/*)

auth (login, logout, me), dashboard, users (+ bookings), movies, theaters, screens, shows, seats (+ batch), movie-bookings (+ cancel), bus-operators, buses (+ seats), routes, trips, cities, boarding-points, dropping-points, bus-bookings (+ cancel), payments, refunds, audit-logs. Each resource follows the pattern *index* → *list/create*, *[id]/index* → *get/update/delete*, delegating to AdminXService.

5. Data Model

5.1 Movie & identity domain

Table	Role
login_auth	Users (name, email, phone, password_hash, role, is_active)
refresh_tokens / password_reset_tokens	Hashed tokens with expiry
cities, theaters, screens, seats	Venue hierarchy; seats carry tier + price multiplier
movies, banners, trailers	Catalogue and home content (status, slug indexes)
shows	Movie × screen × time with base price
seat_locks	Temporary holds (lock_token, expires_at)
bookings, booking_seats	Confirmed bookings with code, totals, QR data

5.2 Bus domain (db/04_bus_booking_schema.sql)

Table	Role
bus_operators, buses, bus_seats	Fleet; seats have window / aisle / deck / gender preference

Table	Role
bus_routes, bus_trips	Source→destination, travel date and times
boarding_points, dropping_points	Pickup / drop locations
bus_seat_locks	10-minute holds
bus_promos	Coupon definitions
bus_bookings, bus_booking_passengers, bus_booking_seats	Booking header (code + PNR), passengers, seat map
bus_payments	Payment intents and verification
bus_cancellations, bus_refunds	Cancellation record and refund tracking

5.3 Tables created at runtime / by tools

user_royal_pass, user_cinepoints, cinepoint_transactions (WalletService._ensureTables), **wallets / wallet_transactions**, and **audit_logs, movie_payments, movie_refunds** (tool/setup_admin_schema.dart). Schema is therefore split between SQL files, runtime DDL and scripts – there is no single migration history.

6. Core Business Logic

6.1 Authentication & sessions

- Register hashes the password with bcrypt; login looks up by email (lower-cased) or phone and issues a 15-min HS256 JWT plus a 30-day opaque refresh token (SHA-256 hash stored in refresh_tokens).
- Password reset uses a similar random token stored hashed in password_reset_tokens (expiry from JWT_RESET_MINUTES).
- User routes verify the Bearer token in each service (AuthUtils.verifyAccessToken); there is no global auth middleware.
- Admin routes re-check the DB on every call: user must have role admin or super_admin and is_active = true.

6.2 Movie booking flow

- **Seat map:** expired locks are deleted, then seats are returned as available / locked / booked.
- **Lock:** 8-minute hold written to seat_locks, returns a lockToken (rejects seats already locked/booked).
- **Create booking:** validates lockToken, prices each seat (base price × tier multiplier), applies automatic 20% Royal Pass discount for active premium users, adds ■35.40 convenience fee, generates code TIC-YYYY-xxxxxxx, builds a base64url QR payload, deletes the locks, and awards 5 CinePoints per ticket.

6.3 Bus booking flow

- Search by city pair and date; seat layout shows deck, window/aisle and gender-aware booked state (women-reserved logic).
- **Recommend seats:** rule-based scoring by preference (comfort / window / etc.) returning ranked seats with score and reason text – labelled "AI" in the docs but implemented as heuristics.
- **Hold** 10 minutes; **price** = base fare + 5% GST + ■25 convenience fee – promo discount (percentage with max cap, or flat; min booking amount enforced).
- **Payment:** create intent (payment_id PAY-BUS-..., UPI intent string, gateway options) → verify with transaction reference → booking confirmed with PNR.
- **Cancellation refund** by hours before departure:

Hours to departure	Refund %	Charge
≥ 24	90%	10%
12 – 24	75%	25%
6 – 12	50%	50%
< 6	25%	75%

6.4 Wallet, CinePoints & Royal Pass

- Wallet balance with transaction ledger; top-up and pay endpoints.
- CinePoints: 5 per movie ticket; redeemable at ■0.50 per point into wallet balance.
- Royal Pass: 365-day premium (extends from current expiry on renewal), 20% discount on movie bookings, purchased from wallet balance.

7. Build, Deploy & Operations

- **Run locally:** `dart_frog dev` (generates server) or `dart run bin/server.dart`; port from `PORT` (default 8080).
- **Docker:** multi-stage build, AOT `dart compile exe`, small Debian runtime with CA certificates (needed for TLS to managed Postgres).
- **Config:** `DB_HOST/PORT/NAME/USER/PASSWORD`, `JWT_SECRET`, `JWT_ISSUER`, `JWT_ACCESS_MINUTES`, `JWT_REFRESH_DAYS`, `JWT_RESET_MINUTES`.
- **Observability:** stdout logs with per-request latency; no metrics, tracing or health endpoint.
- **Testing:** one mocktail-based middleware test; `tool/test_api.dart`, `test_admin_apis.dart`, `test_bus_apis.dart` are manual integration smoke scripts against `localhost:8080`.

8. Findings & Risks

Observations from reading the code, ordered by severity. These are analysis notes, not changes made to the project.

#	Severity	Finding	Recommendation
1	High	.env is tracked in git (listed by <code>git ls-files</code> , currently modified) and holds DB and JWT credentials; <code>.env</code> appears in several past commits.	Remove from index, add to <code>.gitignore</code> , rotate DB password and JWT secret, use platform secrets.
2	High	Payment verification is not authoritative: <code>bus /verify</code> accepts a client-supplied reference; no gateway signature check. A placeholder key <code>rzp_live_...</code> is returned in responses.	Integrate real gateway, verify HMAC signature / webhook server-side.
3	High	Anonymous fallback in bus payment: if no valid user, payment is attributed to the first user row in <code>login_auth</code> .	Require authentication; reject instead of defaulting.
4	High	No DB transactions or row locks: seat locking/booking performs check-then-insert across separate statements; wallet debit/credit and booking inserts are not atomic.	Wrap in transactions, <code>SELECT ... FOR UPDATE</code> , unique constraints on (<code>show</code> , <code>seat</code>).
5	Medium	New PostgreSQL connection per request (no pool) – high latency and connection exhaustion under load.	Use postgres Pool, one pool per process.
6	Medium	CORS allows <code>*</code> ; no rate limiting seen on login / password reset.	Restrict origins, add throttling.
7	Medium	JWT lifetime hard-coded to 15 min and refresh to 30 days while <code>Env.jwtAccessMinutes</code> / <code>jwtRefreshDays</code> are defined but unused.	Use configured values.
8	Medium	Raw exception text (<code>\$e</code>) returned to clients in 500 responses.	Return generic message, log details.
9	Medium	Stale generated file: <code>bin/server.dart</code> is generated output (still references removed <code>/movies/[slug]</code> route); <code>build/</code> is git-ignored.	Do not commit generated files; regenerate on build.
10	Low	Schema split across SQL files, runtime <code>CREATE TABLE</code> and tools; 55k-line dump (<code>db/04-10-26</code>) in repo; crypto not declared in <code>pubspec</code> (used transitively).	Adopt versioned migrations; add crypto dependency explicitly; keep dumps out of git.
11	Low	Minimal tests (middleware only); services are static and hard to mock.	Introduce repository layer / DI and service tests.
12	Low	Mixed response envelopes (<code>{error}</code> vs <code>{status, message}</code>) and inconsistent route naming (<code>api/admin</code> prefix vs top level).	Standardise envelope.

9. Recommended Next Steps

- Immediately: rotate secrets, untrack `.env`, remove history exposure.

- Short term: connection pool, transactional booking/payment/wallet flows, real payment verification, auth middleware for user routes.
- Medium term: versioned migrations, health/metrics endpoints, integration tests with a disposable Postgres, OpenAPI spec generated from routes.
- Longer term: caching for catalogue/home feed, background job for expired-lock cleanup instead of cleaning on read.